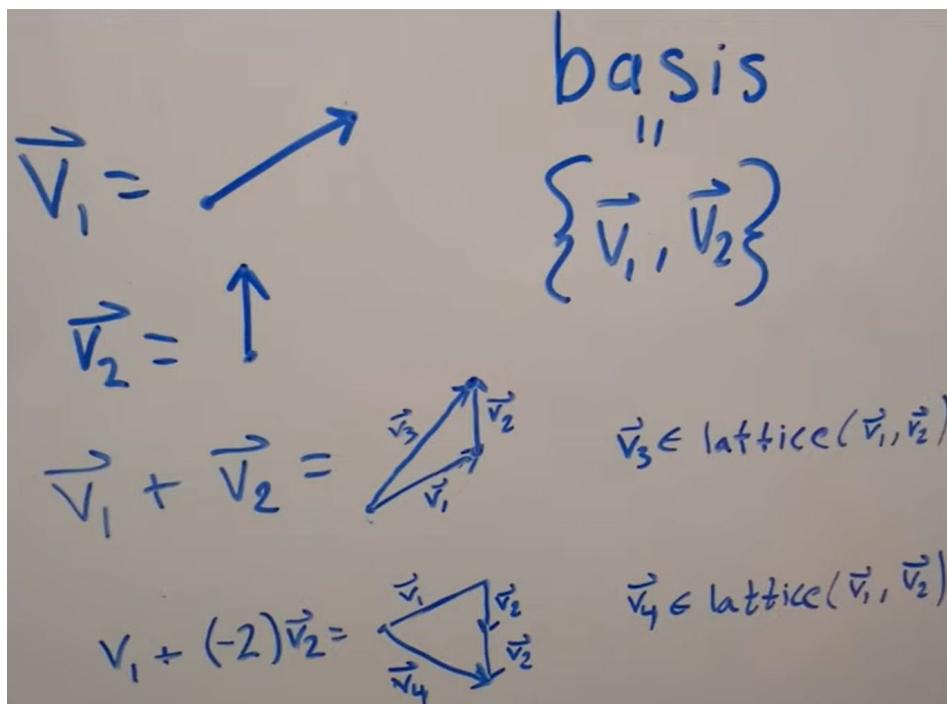


From: [Rahman Ziaur](#)
To: [Ziaur Rahman , 124606](#)
Subject: Lattice based Signature with Learning With Error (LWE) for Postquantum Computing
Date: Friday, 18 August 2023 12:33:28 PM
Attachments: [image.png](#)
[image.png](#)
[image.png](#)
[image.png](#)
[image.png](#)
[image.png](#)
[image.png](#)
[image.png](#)
[image.png](#)
[image.png](#)
[image.png](#)
[image.png](#)
[image.png](#)
[image.png](#)
[image.png](#)
[image.png](#)
[image.png](#)

Hi There,

Here is the simple illustration how Lattice-based signature works.

Given n linearly independent vectors $b_1, b_2, \dots, b_n \in \mathbb{R}^m$ the lattice generated by them is defined as –
Lattice, $L(b_1, b_2, b_3 \dots b_n) = \sum x_i b_i \mid x_i \in \mathbb{Z}$

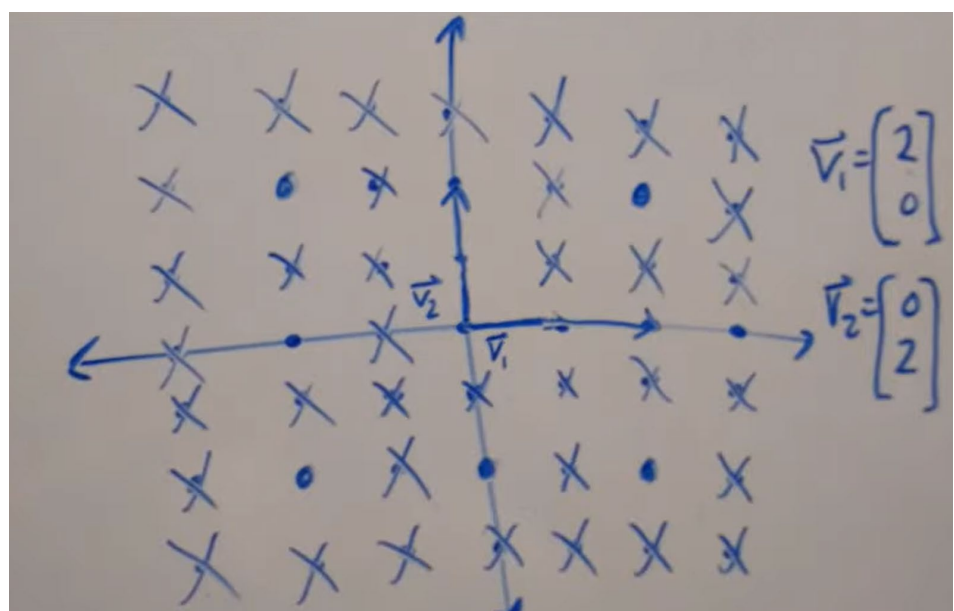
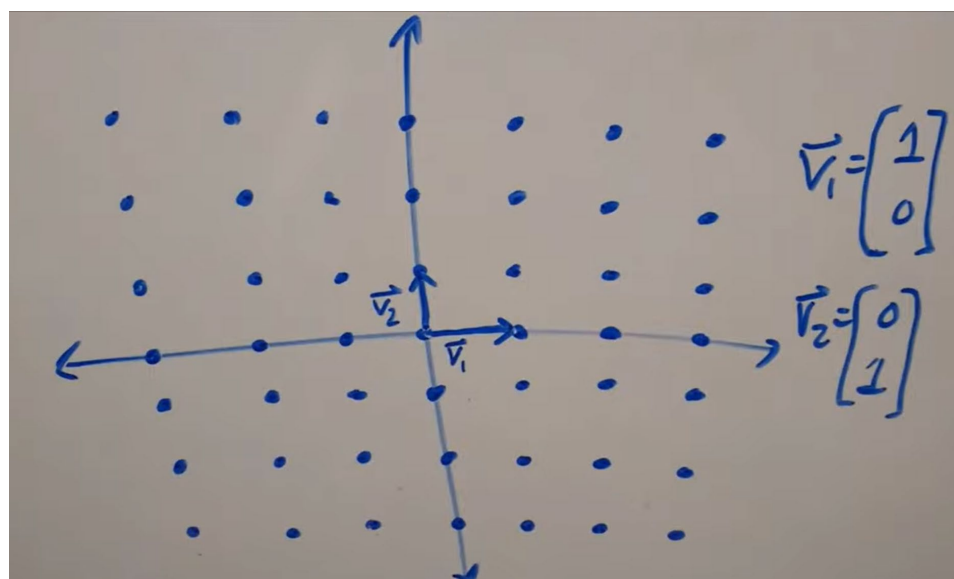


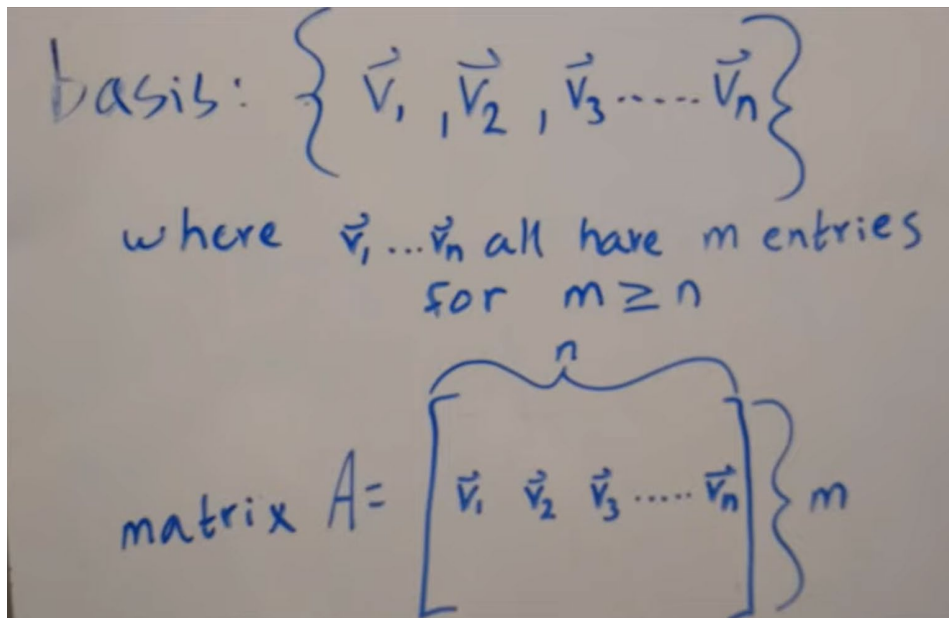
basis

$$\vec{v}_1 = \begin{bmatrix} a_1 \\ a_2 \\ \vdots \\ a_i \end{bmatrix} \quad \vec{v}_2 = \begin{bmatrix} b_1 \\ b_2 \\ \vdots \\ b_i \end{bmatrix} \quad \{\vec{v}_1, \vec{v}_2\}$$

then $a_j, b_j \in \mathbb{Z}$
for $1 \leq j \leq i$

$$\vec{v} = x \vec{v}_1 + y \vec{v}_2 \quad \text{where } \vec{v} \in \text{lattice} \text{ if } x, y \in \mathbb{Z}$$





Shortest Vector Problem: Find the shortest non-zero vector in a lattice given the basis of that lattice

Closest Vector Problem: Given a basis of a lattice and a vector, v , *not* in the lattice, find the closest vector to v that is in the lattice

Bounded Distance Decoding: Similar to the Closest Vector Problem, find the lattice point, s , closest given that v is known to be close to s

Covering Radius Problem: Given a basis for a lattice find the smallest sphere that when placed at every lattice point it includes 2 lattice points

In other ways -

Shortest vector problem (SVP). The *SVP* is to find $x \in \Lambda(b_1, \dots, b_n) - \{0\}$ such that $\|x\|$ is smallest. Note that this can be in any metric. The only known algorithms are all in exponential time.

Shortest independent vector problem (SIVP). In the *SIVP*, we seek to minimize the length of the longest vector in the basis. In other words, we seek to find a new basis which yields the same lattice, but minimizes the length of the longest vector.

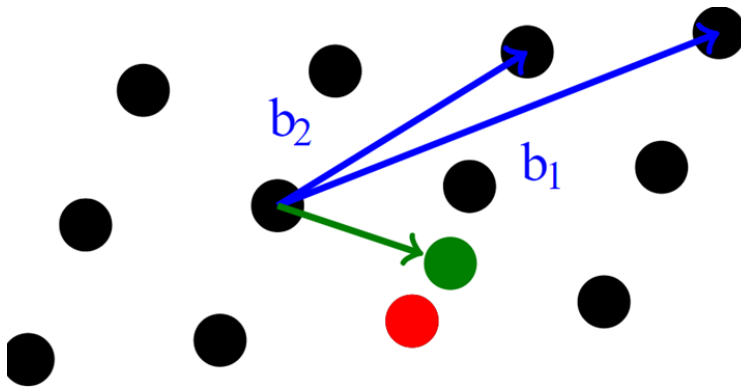
Dual lattice. We define the *dual lattice* Λ^* of lattice Λ , as the set of vectors whose inner product yields an integer for all points in the lattice. That is,

$$\Lambda^* = \{y \in \mathbf{R}^n \mid x^T y \in \mathbb{Z} \quad \forall x \in \Lambda\}$$

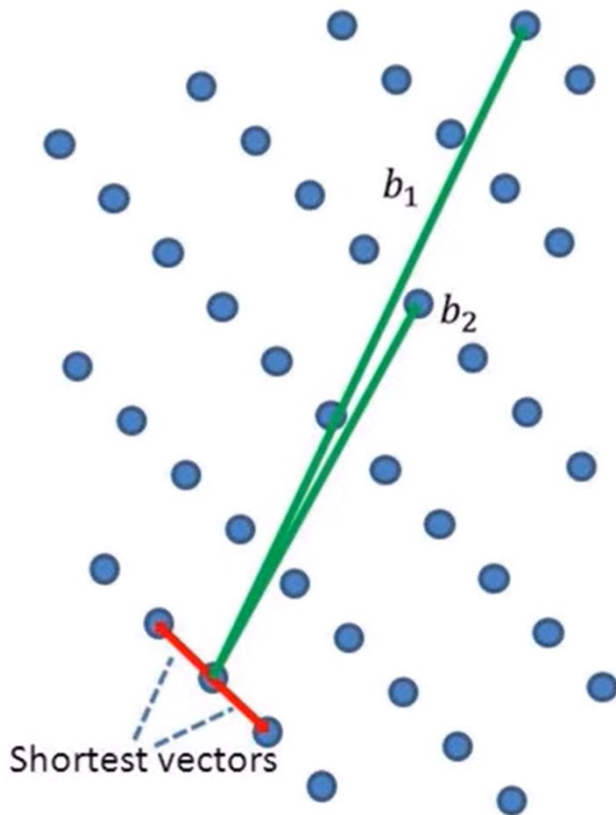
Covering radius problem (CRP): Given a basis for the lattice, the algorithm must find the largest distance (or in some versions, its approximation) from any vector to the lattice.

Hardness of SVP implies the same hardness for CVP [Goldreich et al]

Finding Shortest/Closest Vector:



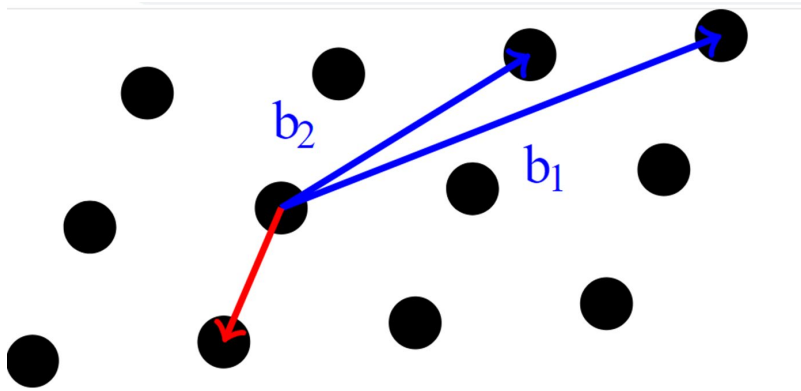
CVP: closest vector problem (basis vectors in blue, external vector in green, closest vector in red).



Python Example:

```
from scipy import spatial
A = [[0,1,2,3,4], [4,3,2,1,0], [2,5,3,7,1], [1,0,1,0,1]]
tree = spatial.KDTree(A)
print(A[tree.query([0.5,0.5,0.5,0.5,0.5])[1]])
```

[1, 0, 1, 0, 1]

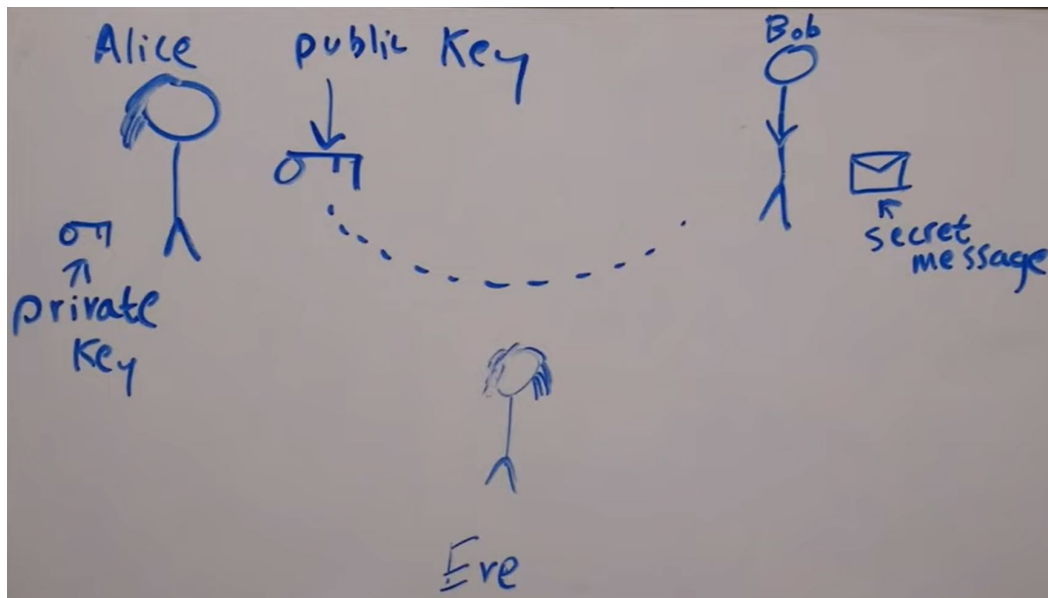


SVP: shortest vector problem (basis vectors in blue, shortest vector in red)

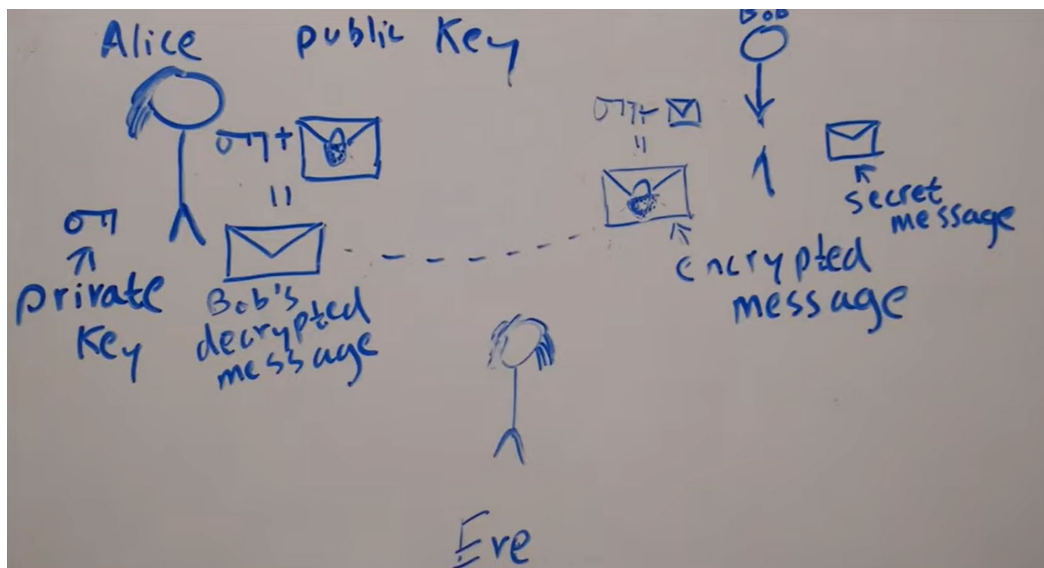
The closest vector problem is a generalization of the shortest vector problem. It is easy to show that given an oracle for CVP (defined below), one can solve SVP by making some queries to the oracle. The naive method to find the shortest vector by calling the CVP oracle to find the closest vector to 0 does not work because 0 is itself a lattice vector and the algorithm could potentially output 0.

Now Let's see how this concept can be applied in the asymmetric key cryptography-based signature mechanism.

Let Bob wants to send secret message to Alice after signing it using his public key of the Lattice-based signature.



Alice decrypts it using her private key.



First of all, Alice & Bob chooses a random Matrix $A^{(m \times n)}$ over a fairly large integer. Alice chooses a private binary vector x where x is unique because each of its m entries is either 0 or 1. She computes Ax and publishes here public key $U = Ax$. Finding x is impossible as it's a Collision resistant hash function.

public: A and q

Alice & Bob chooses a random Matrix $A^{(m \times n)}$

$$A^{m \times n} = \begin{bmatrix} a_{11} & a_{12} & \dots & a_{1n} \\ a_{21} & \dots & \dots & \dots \\ \vdots & \dots & \dots & \dots \\ a_{m1} & \dots & \dots & a_{mn} \end{bmatrix} \text{ where } a_{ij} < q \text{ and } a_{ij} \in \mathbb{Z}$$

Alice private: $\vec{x}$ (Private Vector)

Alice public: $\vec{u} = A\vec{x}$ Alice sends to Bob

$$\vec{x} \stackrel{?}{=} A^{-1}\vec{u}$$

Collision Resistant Hash Function

Bob computes vector b_1, b_2 . He selects secret vector S with m entries. He calculates, $b_1 = As + e_1$ where e_1 is another yet random vector with m integers except the magnitude is very small. He then computes $b_2 = SU + e_2 + \text{bit}(q/2)$, where e_2 has the same description but different values. Then Bob sends an encrypted message to Alice using PubKey.

Bob

Bob sends an encrypted message to Alice using PubKey

public: $\vec{b}_1, b_2$ $\vec{b}_1 = A\vec{s} + \vec{e}_1$

private: $\vec{s}, \vec{e}_1, e_2$ $b_2 = \vec{s} \cdot \vec{u} + e_2 + \text{bit} \cdot \frac{q}{2}$

bit = 0 or 1 $e \ll \frac{q}{4}$

Decryption:

Alice can find if Bob sends 0 or 1

Decryption

$\vec{b}_1 \cdot \vec{x} = \vec{s} \cdot \vec{u} + \vec{e}_1 \cdot \vec{x}$

$$b_2 - \vec{b}_1 \cdot \vec{x} = \vec{s} \cdot \vec{u} + e_2 + \text{bit} \cdot \frac{q}{2} - \vec{s} \cdot \vec{u} + \vec{e}_1 \cdot \vec{x}$$

$$(e_2 - \vec{e}_1 \cdot \vec{x}) + \text{bit} \cdot \frac{q}{2}$$

$$(e_2 - \vec{e}_1 \cdot \vec{x}) + \text{bit} \cdot \frac{q}{2}$$

very small only small if bit=0

If Bob sends 0 then result will be closed to 0 Or if he sends 1 then result will be closed to 1 because of error vector with the magnitude closed to 0. That's how error vector helps learning the correct message.

Ref: <https://www.youtube.com/watch?v=37Ri1jpl5p8>

